

Write a program to find the minimal length of a contiguous subarray for which the sum $\geq S$.

Sample Input:

Enter number of elements: 6

Enter elements: 2 3 1 2 4 3

Enter value of S: 7

Sample Output:

Minimum length subarray: 2

Your Code

```
1 include <stdio.h>
2
3 int main() {
4     int n;
5     printf("Enter number of elements: ");
6     scanf("%d", &n);
7
8     int arr[n];
9     printf("Enter elements: ");
10    for (int i = 0; i < n; i++) {
11        scanf("%d", &arr[i]);
12    }
13
14    int S;
15    printf("Enter value of S: ");
16    scanf("%d", &S);
17
18    int minLength = n + 1;
19    int windowSum = 0;
```

Program Input (stdin)

Enter input if required...

Output

```
Enter number of elements: Enter elements:
Enter value of S: Minimum length subarray:
3
```

Run

Save

Define a structure named Point with members x and y representing coordinates. Write a program to read the coordinates of a point, calculate the distance from the origin (0,0), and determine the quadrant in which the point lies. Finally, display the coordinates, distance from the origin, and the quadrant.

Sample Input: Enter x-coordinate: 3 Enter y-coordinate: 5

Sample Output:

Point Coordinates: x = 3, y = 5 Distance from origin: 5.83 Quadrant: 1

Explanation:

Distance = $\sqrt{(x - 0)^2 + (y - 0)^2}$

So, for the given point with coordinates (3, 5), the distance from the origin is calculated as follows:

Distance = $\sqrt{(3 - 0)^2 + (5 - 0)^2} = \sqrt{3^2 + 5^2} = \sqrt{9 + 25} = \sqrt{34} \approx 5.83$

our Code

```
1 #include <stdio.h>
2 #include <math.h>
3
4 struct Point {
5     int x;
6     int y;
7 };
8
9 int main() {
10     struct Point p;
11     double distance;
12     int quadrant;
13
14     printf("Enter x-coordinate: ");
15     scanf("%d", &p.x);
16     printf("Enter y-coordinate: ");
17     scanf("%d", &p.y);
```

Program Input (stdin)

Enter input if required...

Output

```
Enter x-coordinate: Enter y-coordinate:
Point Coordinates: x = 4567, y = 32759
Distance from origin: 33075.82
Quadrant: 1
```

▶ Run

📁 Save

Write a program to check if a given string is a palindrome.

Sample Input:

Enter a string: madam

Sample Output:

The string is a palindrome.

Your Code

```
1 #include <stdio.h>
2 #include <string.h>
3
4 int main() {
5     char s[1000];
6     int i, n, isPal = 1;
7
8     printf("Enter a string: ");
9     scanf("%999s", s);
10
11     n = strlen(s);
12
13     for (i = 0; i < n / 2; i++) {
14         if (s[i] != s[n - 1 - i]) {
15             isPal = 0;
16             break;
17         }
18     }
19
20     if (isPal)
21         printf("The string is a palindrome.\n");
```

Program Input (stdin)

Enter input if required...

Output

Enter a string: The string is a palindrome.

▶ Run

📁 Save

Write a program to find the maximum element in each row of a matrix.

Sample Input:

Enter number of rows: 3

Enter number of columns: 3

Enter elements:

1 2 3

4 5 6

7 8 9

Sample Output:

Maximum in each row: 3 6 9

Your Code

```
1 #include <stdio.h>
2
3 int main() {
4     int rows, cols, i, j;
5     int matrix[100][100];
6
7     // Input number of rows and columns
8     printf("Enter number of rows: ");
9     scanf("%d", &rows);
10    printf("Enter number of columns: ");
11    scanf("%d", &cols);
12
13    // Input matrix elements
14    printf("Enter elements:\n");
15    for (i = 0; i < rows; i++) {
16        for (j = 0; j < cols; j++) {
17            scanf("%d", &matrix[i][j]);
18        }
19    }
```

Program Input (stdin)

Enter input if required...

Output

```
Enter number of rows: Enter number of
columns: Enter elements:
Maximum in each row:
```

▶ Run

📁 Save